

Tìm kiếm nhị phân (Binary Search)

Nội dung

Trong bài học này, chúng ta sẽ tìm hiểu một trong những kỹ thuật quan trọng nhất trong lập trình thi đấu: Tìm kiếm nhị phân (Binary Search). Đây là kỹ thuật giúp bạn tìm kiếm cực kỳ nhanh — thay vì kiểm tra từng phần tử một, ta có thể loại bỏ một nửa dữ liệu sau mỗi bước.

1. Tại sao cần tìm kiếm nhị phân?
 - Tìm kiếm tuyến tính (Linear Search) — cách "thô"
 - Tìm kiếm nhị phân (Binary Search) — cách thông minh
2. Tìm phần tử trong mảng đã sắp xếp
 - Thuật toán
 - Ví dụ trực quan
 - Code
3. Lower Bound và Upper Bound
 - Ví dụ minh họa
 - Cách sử dụng
 - Các ứng dụng thường gặp
4. Chặt nhị phân đáp án (Binary Search on Answer)
 - Ý tưởng cốt lõi
 - Tính đơn điệu là gì?
 - Dạng code tổng quát
 - Ví dụ chi tiết: Bài "Factory Machines"(CSES 1620)
 - Code hoàn chỉnh
 - Mẹo nhận biết bài Binary Search on Answer
5. Ví dụ bổ sung: Chia mảng thành k đoạn
 - Hàm `check(S)`
6. Sai lầm thường gặp
 - Sai lầm 1: Vòng lặp vô tận
 - Sai lầm 2: Chọn biên sai
 - Sai lầm 3: Quên kiểu số lớn
 - Sai lầm 4: Nhầm hướng binary search
7. Tóm tắt

1. Tại sao cần tìm kiếm nhị phân?

Tìm kiếm tuyến tính (Linear Search) — cách "thô"

Giả sử bạn có một mảng n phần tử và cần tìm xem số x có trong mảng hay không. Cách đơn giản nhất là duyệt từ đầu đến cuối:

C++

```
bool found = false;
for (int i = 0; i < n; i++) {
    if (a[i] == x) {
        found = true;
        break;
    }
}
```

Python

```
found = False
for value in a:
    if value == x:
        found = True
        break
```

Cách này có độ phức tạp $O(n)$. Nếu $n = 10^6$, ta cần đến 1 triệu phép so sánh trong trường hợp xấu nhất. Với 1 truy vấn thì ổn, nhưng nếu có $q = 10^5$ truy vấn thì tổng cộng là 10^{11} phép tính — quá chậm.

Tìm kiếm nhị phân (Binary Search) — cách thông minh

Ý tưởng cốt lõi: Nếu mảng đã được sắp xếp, ta có thể tìm kiếm nhanh hơn rất nhiều.

Hãy nghĩ về ví dụ thực tế: Khi bạn tra từ điển (sách giấy), bạn có dò từ trang 1 đến trang cuối không? Không. Bạn sẽ:

1. Mở giữa cuốn từ điển.
2. Nếu từ cần tìm nằm trước trang hiện tại → lật sang nửa trái.
3. Nếu từ cần tìm nằm sau trang hiện tại → lật sang nửa phải.
4. Lặp lại cho đến khi tìm thấy.

Đây chính là tìm kiếm nhị phân. Mỗi bước ta loại bỏ một nửa phần còn lại. Với mảng $n = 10^6$ phần tử, ta chỉ cần khoảng $\log_2(10^6) \approx 20$ bước. 20 bước thay vì 1 triệu bước.

SỐ PHẦN TỬ N	LINEAR SEARCH (XẤU NHẤT)	BINARY SEARCH (XẤU NHẤT)
1,000	1,000 bước	10 bước
1,000,000	1,000,000 bước	20 bước
10^9	10^9 bước	30 bước

2. Tìm phần tử trong mảng đã sắp xếp

Thuật toán

Cho mảng a đã sắp xếp tăng dần và số x cần tìm. Ta duy trì hai biến:

- lo : chỉ số trái
- hi : chỉ số phải

Ban đầu $lo = 0$, $hi = n - 1$. Tại mỗi bước:

1. Tính mid — vị trí giữa.
2. Nếu $a[mid] == x \rightarrow$ tìm thấy.
3. Nếu $a[mid] < x \rightarrow x$ nằm ở nửa phải $\rightarrow lo = mid + 1$.
4. Nếu $a[mid] > x \rightarrow x$ nằm ở nửa trái $\rightarrow hi = mid - 1$.
5. Nếu $lo > hi \rightarrow$ không tìm thấy.

Ví dụ trực quan

Cho mảng $a = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ (10 phần tử). Tìm $x = 23$.

Bước 1: $lo = 0$, $hi = 9$, $mid = 4$.

Chỉ số:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
Giá trị:	2	5	8	12	16	23	38	56	72	91
					lo	mid				hi

$a[4] = 16 < 23 \rightarrow x$ nằm bên phải $\rightarrow lo = 5$.

Bước 2: $lo = 5$, $hi = 9$, $mid = 7$.

Chỉ số:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
Giá trị:	2	5	8	12	16	23	38	56	72	91
						lo	mid			hi

$a[7] = 56 > 23 \rightarrow x$ nằm bên trái $\rightarrow hi = 6$.

Bước 3: $lo = 5$, $hi = 6$, $mid = 5$.

Chỉ số:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
Giá trị:	2	5	8	12	16	23	38	56	72	91
						lo/mid	hi			

$a[5] = 23 == 23 \rightarrow$ Tìm thấy. Chỉ mất 3 bước thay vì 6 bước nếu dò tuyến tính.

Code

C++

```

int lo = 0, hi = n - 1;
bool found = false;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2; // tránh tràn số
    if (a[mid] == x) {
        found = true;
        break;
    } else if (a[mid] < x) {
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}

```

Python

```

lo, hi = 0, n - 1
found = False
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == x:
        found = True
        break
    elif a[mid] < x:
        lo = mid + 1
    else:
        hi = mid - 1

```

Lưu ý:

- Trong C++, nên viết $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ thay vì $\text{mid} = (\text{lo} + \text{hi}) / 2$ để tránh tràn số.
- Trong Python thì số nguyên không bị tràn theo kiểu `int` của C++, nhưng bạn vẫn có thể dùng cùng công thức để giữ thói quen nhất quán.

3. Lower Bound và Upper Bound

Trong thực tế, ta không chỉ muốn biết “có hay không”, mà còn muốn biết vị trí phần tử trong mảng đã sắp xếp.

- `lower_bound(x)`: vị trí đầu tiên có giá trị $\geq x$.
- `upper_bound(x)`: vị trí đầu tiên có giá trị $> x$.

Ví dụ minh họa

Mảng a : [1, 3, 3, 3, 5, 7, 9] (chỉ số: 0, 1, 2, 3, 4, 5, 6).

HÀM	$x = 3$	$x = 4$	$x = 0$	$x = 10$
lower_bound	chỉ số 1 (giá trị 3)	chỉ số 4 (giá trị 5)	chỉ số 0 (giá trị 1)	chỉ số 7 (quá cuối mảng)
upper_bound	chỉ số 4 (giá trị 5)	chỉ số 4 (giá trị 5)	chỉ số 0 (giá trị 1)	chỉ số 7 (quá cuối mảng)

Cách sử dụng

C++

```
vector<int> a = {1, 3, 3, 3, 5, 7, 9};
auto it = lower_bound(a.begin(), a.end(), 3);
int pos = it - a.begin(); // pos = 1

auto it2 = upper_bound(a.begin(), a.end(), 3);
int pos2 = it2 - a.begin(); // pos2 = 4
```

Python

```
from bisect import bisect_left, bisect_right
a = [1, 3, 3, 3, 5, 7, 9]
pos = bisect_left(a, 3) # 1
pos2 = bisect_right(a, 3) # 4
```

Các ứng dụng thường gặp

1. Đếm số phần tử $= x$:

C++

```
int count_x = upper_bound(a.begin(), a.end(), x)
               - lower_bound(a.begin(), a.end(), x);
```

Python

```
count_x = bisect_right(a, x) - bisect_left(a, x)
```

2. Đếm số phần tử $\leq x$:

C++

```
int count_le = upper_bound(a.begin(), a.end(), x) - a.begin();
```

Python

```
count_le = bisect_right(a, x)
```

3. Đếm số phần tử $< x$:

C++

```
int count_lt = lower_bound(a.begin(), a.end(), x) - a.begin();
```

Python

```
count_lt = bisect_left(a, x)
```

4. Đếm số phần tử trong đoạn $[l, r]$:

C++

```
int count_range = upper_bound(a.begin(), a.end(), r)
                  - lower_bound(a.begin(), a.end(), l);
```

Python

```
count_range = bisect_right(a, r) - bisect_left(a, l)
```

Mẹo nhớ:

- $\text{lower_bound}(x)$ / $\text{bisect_left}(a, x)$ là vị trí chèn vào bên trái các phần tử bằng x .
- $\text{upper_bound}(x)$ / $\text{bisect_right}(a, x)$ là vị trí chèn vào bên phải các phần tử bằng x .

4. Chặt nhị phân đáp án (Binary Search on Answer)

Đây là dạng mạnh nhất và phổ biến nhất của binary search trong competitive programming.

Ý tưởng cốt lõi

Thay vì tìm kiếm trong mảng, ta tìm kiếm trên miền giá trị của đáp án:

- Đề bài hỏi: “Tìm giá trị nhỏ nhất hoặc lớn nhất sao cho một điều kiện nào đó đúng”.
- Ta nhận thấy điều kiện này có tính đơn điệu.
- Khi đó, ta có thể dùng binary search để tìm ranh giới.

Tính đơn điệu là gì?

Hãy tưởng tượng với mỗi giá trị k , ta đánh dấu:

- NO nếu chưa thỏa.
- OK nếu đã thỏa.

Ví dụ:

```
k = 1 2 3 4 5 6 7 8 9 10
    NO NO NO NO OK OK OK OK OK
```

Nếu tất cả NO nằm một phía và tất cả OK nằm phía còn lại, thì ta có thể dùng binary search. Nếu kết quả lộn xộn kiểu NO, OK, NO, OK thì không dùng được.

Dạng code tổng quát

Tìm giá trị nhỏ nhất thỏa mãn:

C++

```
int lo = /* giá trị nhỏ nhất có thể */;
int hi = /* giá trị lớn nhất có thể */;
int ans = hi;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (check(mid)) {
        ans = mid;
        hi = mid - 1;
    } else {
        lo = mid + 1;
    }
}
```

Python

```
lo = ...
hi = ...
ans = hi
while lo <= hi:
    mid = (lo + hi) // 2
    if check(mid):
        ans = mid
        hi = mid - 1
    else:
        lo = mid + 1
```

Tìm giá trị lớn nhất thỏa mãn:

C++

```
int lo = /* giá trị nhỏ nhất có thể */;
int hi = /* giá trị lớn nhất có thể */;
int ans = lo;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (check(mid)) {
        ans = mid;
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}
```

Python

```
lo = ...
hi = ...
ans = lo
while lo <= hi:
    mid = (lo + hi) // 2
    if check(mid):
        ans = mid
        lo = mid + 1
    else:
        hi = mid - 1
```

Ví dụ chi tiết: Bài "Factory Machines"(CSES 1620)

Đề bài: Có n máy sản xuất. Máy thứ i cần t_i giây để sản xuất 1 sản phẩm. Hỏi thời gian ít nhất để sản xuất đủ k sản phẩm.

Ví dụ: $n = 3, k = 7, t = [3, 2, 5]$.

Phân tích:

- Nếu thử $T = 6$:
 - Máy 1 làm được $\lfloor 6/3 \rfloor = 2$
 - Máy 2 làm được $\lfloor 6/2 \rfloor = 3$
 - Máy 3 làm được $\lfloor 6/5 \rfloor = 1$

Tổng là $2 + 3 + 1 = 6 < 7 \rightarrow$ chưa đủ.

- Nếu thử $T = 8$:
 - Máy 1 làm được 2
 - Máy 2 làm được 4
 - Máy 3 làm được 1

Tổng là $7 \rightarrow$ đủ.

Ta thấy:

- Nếu T giây đủ, thì $T + 1$ giây cũng đủ.
- Nếu T giây không đủ, thì giá trị nhỏ hơn cũng không đủ.

Đó chính là tính đơn điệu:

T =	1	2	3	4	5	6	7	8	9	10	...
	NO	NO	NO	NO	NO	NO	NO	OK	OK	OK	...
								^			
								đáp án			
								= 8			

Code hoàn chỉnh

C++

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, k;
    cin >> n >> k;
    vector<long long> t(n);
    for (auto &x : t) cin >> x;

    auto check = [&](long long T) -> bool {
        long long total = 0;
        for (int i = 0; i < n; i++) {
            total += T / t[i];
            if (total >= k) return true;
        }
        return false;
    };

    long long lo = 1;
    long long hi = 1LL * (*min_element(t.begin(), t.end())) * k;
    long long ans = hi;

    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (check(mid)) {
            ans = mid;
            hi = mid - 1;
        } else {
            lo = mid + 1;
        }
    }
    cout << ans << '\n';
    return 0;
}

```

Python

```

n, k = map(int, input().split())
t = list(map(int, input().split()))

def check(T):
    total = 0
    for x in t:
        total += T // x
        if total >= k:
            return True
    return False

lo = 1
hi = min(t) * k

```

```

ans = hi

while lo <= hi:
    mid = (lo + hi) // 2
    if check(mid):
        ans = mid
        hi = mid - 1
    else:
        lo = mid + 1

print(ans)

```

Mẹo nhận biết bài Binary Search on Answer

Khi đề bài hỏi các dạng như sau, hãy nghĩ ngay đến binary search on answer:

- Tìm giá trị nhỏ nhất / lớn nhất sao cho...
- Tìm thời gian ít nhất để...
- Có thể đạt được một giá trị nào đó hay không.
- Tối ưu một đáp án số và có thể kiểm tra bằng hàm `check(...)`.

5. Ví dụ bổ sung: Chia mảng thành k đoạn

Bài toán: Cho mảng a gồm n số nguyên dương. Chia mảng thành k đoạn liên tiếp sao cho tổng lớn nhất trong các đoạn là nhỏ nhất có thể.

Ví dụ: $a = [2, 1, 5, 3, 4]$, $k = 3$:

- Chia $[2, 1] \mid [5] \mid [3, 4]$: $\max \text{ tổng} = \max(3, 5, 7) = 7$
- Chia $[2, 1, 5] \mid [3] \mid [4]$: $\max \text{ tổng} = \max(8, 3, 4) = 8$
- Chia $[2] \mid [1, 5] \mid [3, 4]$: $\max \text{ tổng} = \max(2, 6, 7) = 7$

Đáp án tối ưu là 7.

Ta binary search giá trị S = “tổng lớn nhất cho phép của mỗi đoạn”. Nếu một giá trị S là đủ tốt, thì mọi giá trị lớn hơn cũng đủ tốt. Vậy ta có thể binary search trên S .

Hàm `check(S)`

C++

```

bool check(long long S) {
    int groups = 1;
    long long cur_sum = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > S) return false;
        if (cur_sum + a[i] > S) {
            groups++;
            cur_sum = a[i];
        } else {

```

```

        cur_sum += a[i];
    }
}
return groups <= k;
}

```

Python

```

def check(S):
    groups = 1
    cur_sum = 0
    for x in a:
        if x > S:
            return False
        if cur_sum + x > S:
            groups += 1
            cur_sum = x
        else:
            cur_sum += x
    return groups <= k

```

6. Sai lầm thường gặp

Sai lầm 1: Vòng lặp vô tận

Khi viết binary search, phải đảm bảo lo hoặc hi thay đổi sau mỗi bước.

C++

```

// SAI
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (check(mid)) hi = mid;
    else lo = mid;
}

```

Python

```

# SAI
while lo < hi:
    mid = (lo + hi) // 2
    if check(mid):
        hi = mid
    else:
        lo = mid

```

Khi $lo + 1 == hi$ thì $mid = lo$, nên lo không đổi và vòng lặp chạy mãi.

Cách đúng:

C++

```
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (check(mid)) hi = mid;
    else lo = mid + 1;
}
```

Python

```
while lo < hi:
    mid = (lo + hi) // 2
    if check(mid):
        hi = mid
    else:
        lo = mid + 1
```

Sai lầm 2: Chọn biên sai

- Nếu lo quá lớn hoặc hi quá nhỏ, đáp án thật có thể nằm ngoài phạm vi tìm kiếm.
- Hãy chọn biên sao cho chắc chắn đáp án nằm trong đoạn đang xét.

Sai lầm 3: Quên kiểu số lớn

Trong C++, nếu đáp án có thể rất lớn, hãy dùng long long.

C++

```
long long lo = 1, hi = 1e18, ans = -1;
while (lo <= hi) {
    long long mid = lo + (hi - lo) / 2;
}
```

Sai lầm 4: Nhầm hướng binary search

- Tìm nhỏ nhất thỏa mãn $\rightarrow$ nếu `check(mid)` đúng thì đi sang trái.
- Tìm lớn nhất thỏa mãn $\rightarrow$ nếu `check(mid)` đúng thì đi sang phải.

7. Tóm tắt

DẠNG BÀI	KỸ THUẬT	ĐỘ PHỨC TẠP
Tìm phần tử trong mảng đã sort	Binary search cơ bản	$O(\log n)$
Đếm phần tử trong khoảng	lower_bound / upper_bound	$O(\log n)$
Tìm min/max thỏa mãn điều kiện	Binary search on answer	$O(\log(\text{range}) \times T_{\text{check}})$

Quy trình giải bài Binary Search on Answer:

1. Xác định đáp án cần tìm là gì.
2. Kiểm tra tính đơn điệu.
3. Viết hàm `check(mid)`.
4. Chọn biên `lo`, `hi`.
5. Viết vòng lặp binary search.

Chúc các bạn luyện tập vui vẻ! Binary search nhìn đơn giản nhưng rất dễ sai ở chi tiết. Càng luyện nhiều, bạn sẽ càng thấy đây là một công cụ cực kỳ mạnh.

Bài tập